Lab S08: saga orquestada y circuit breaker

Amy Valentina Veraza Garcia | Sistemas Distribuidos 2027-1 | 17 de octubre de 2026

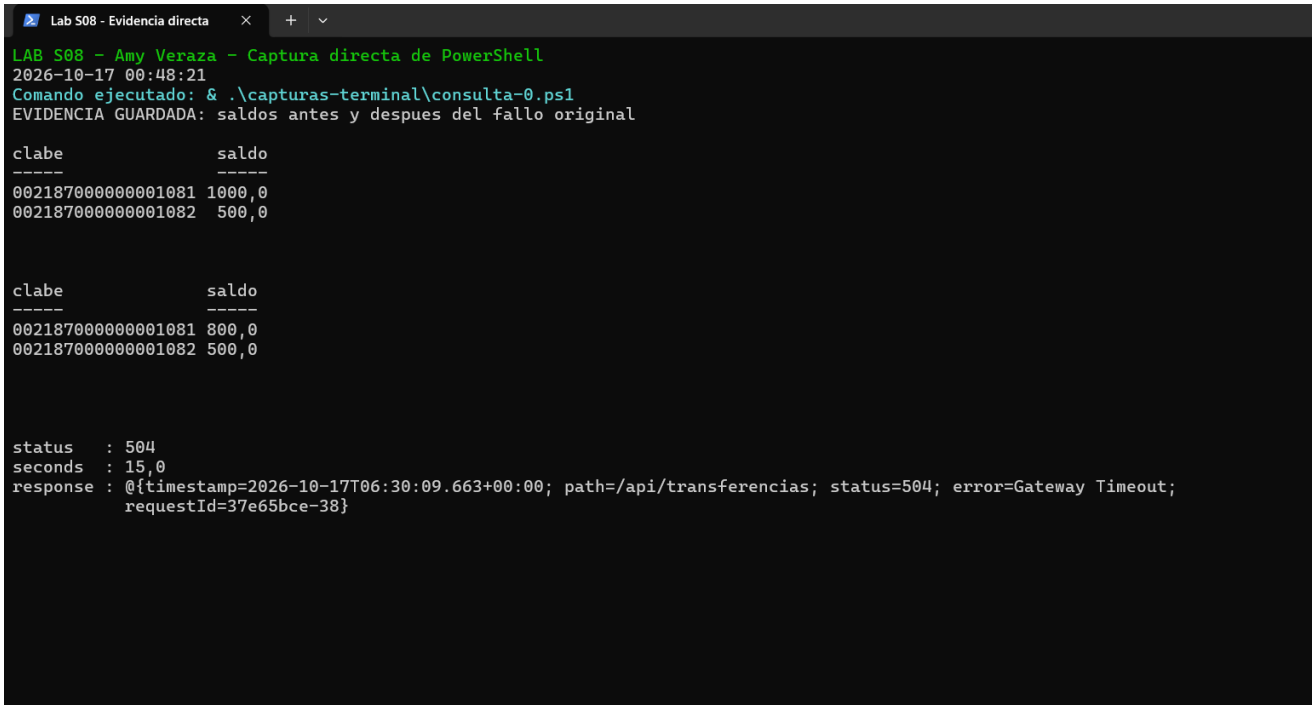
Bitacora con evidencia HTTP, SQL y logs obtenida de contenedores locales. Gateway de prueba: localhost:18080 porque el puerto 8080 estaba ocupado. Se uso una replica de cuenta y bases persistentes. Solo se crearon cuentas ficticias nuevas para aislar los escenarios.

Las capturas son imagenes directas de la ventana de PowerShell, sin redibujar sus resultados. Se ejecutaron consultas HTTP y SQL sobre los contenedores y se consultaron sus logs. Para los saldos anteriores al fallo y los tiempos con cuenta apagada se abrieron los JSON archivados de la prueba original: esas capturas se identifican como evidencia guardada. El diagrama de estados es una figura explicativa.

1. El problema antes de la saga

Con la version original, origen tenia 1000 y destino 500. Transferi 200 y apague cuenta durante una pausa de 20 segundos despues del cargo. Al recuperarla, origen tenia 800 y destino 500. Faltaban 200 en el total observable. La transaccion local de transferencia no revierte las peticiones que cuenta ya confirmo.

El gateway original respondio 504 tras 15 segundos: su timeout era menor que la pausa. El codigo HTTP por si solo no prueba el resultado monetario; consulte ambos saldos despues de recuperar cuenta.



2. Estados de mi saga

Cada paso se guarda mediante una transaccion local del repositorio. El orquestador no mantiene una transaccion de base abierta durante las llamadas HTTP. Se guarda un estado pendiente antes de cada mutacion para identificar operaciones sin confirmacion tras una caida.



Flujo del cargo



Flujo del abono: desde CARGADA



Rechazo del abono: devolver y registrar

CARGADA pasa a ABONO_PENDIENTE. Un rechazo confirmado del abono lleva a COMPENSACION_PENDIENTE; al devolver el dinero pasa a DEVUELTA y, al registrar sus movimientos, a COMPENSADA. Si el abono tiene respuesta incierta, queda pendiente de conciliacion. Una falla del ledger despues de confirmar ambos saldos queda en REGISTRO_PENDIENTE. Un cargo sin confirmacion queda en REVISION_MANUAL.

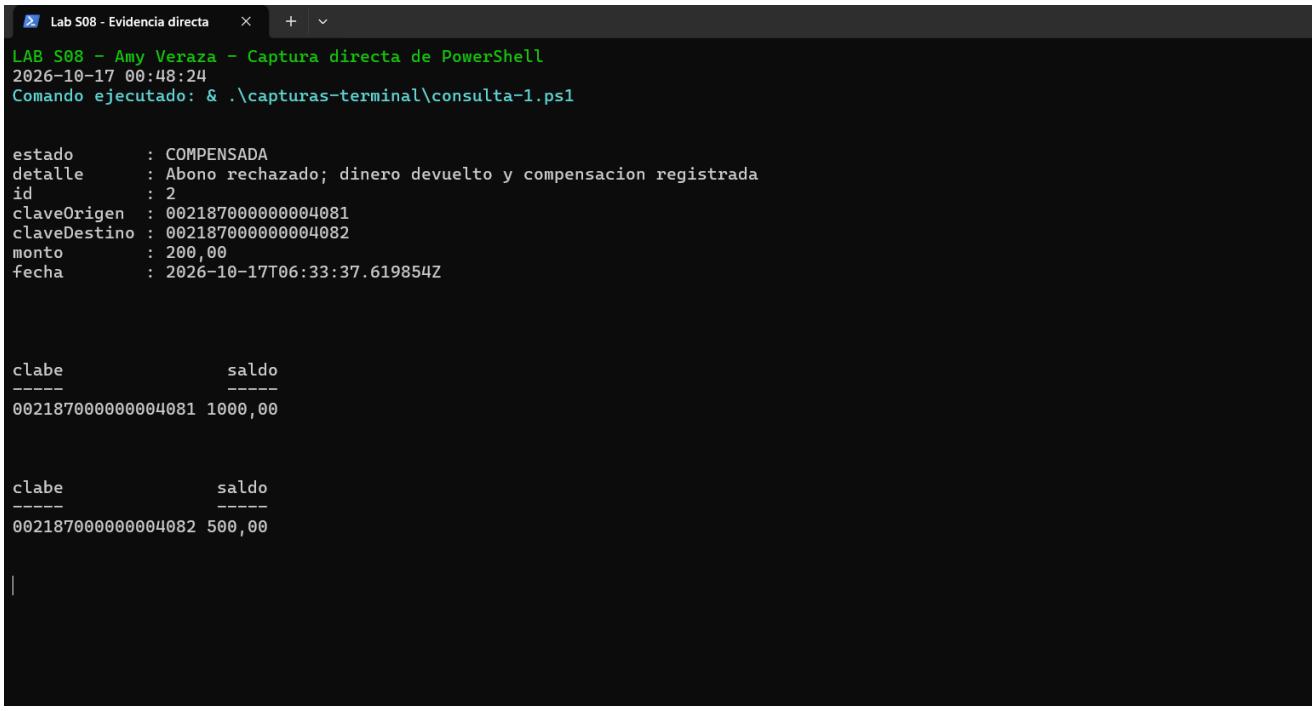
3. Codigo clave

```
paso(t, Estado.CARGADA, "Cargo confirmado");
// Abono rechazado de forma confirmada:
paso(t, Estado.COMPENSACION_PENDIENTE, "Devolucion pendiente");
CuentaRemota devuelta = cuentas.abonar(claveOrigen, monto);
paso(t, Estado.DEVUELTA, "Dinero devuelto; asiento pendiente");
```

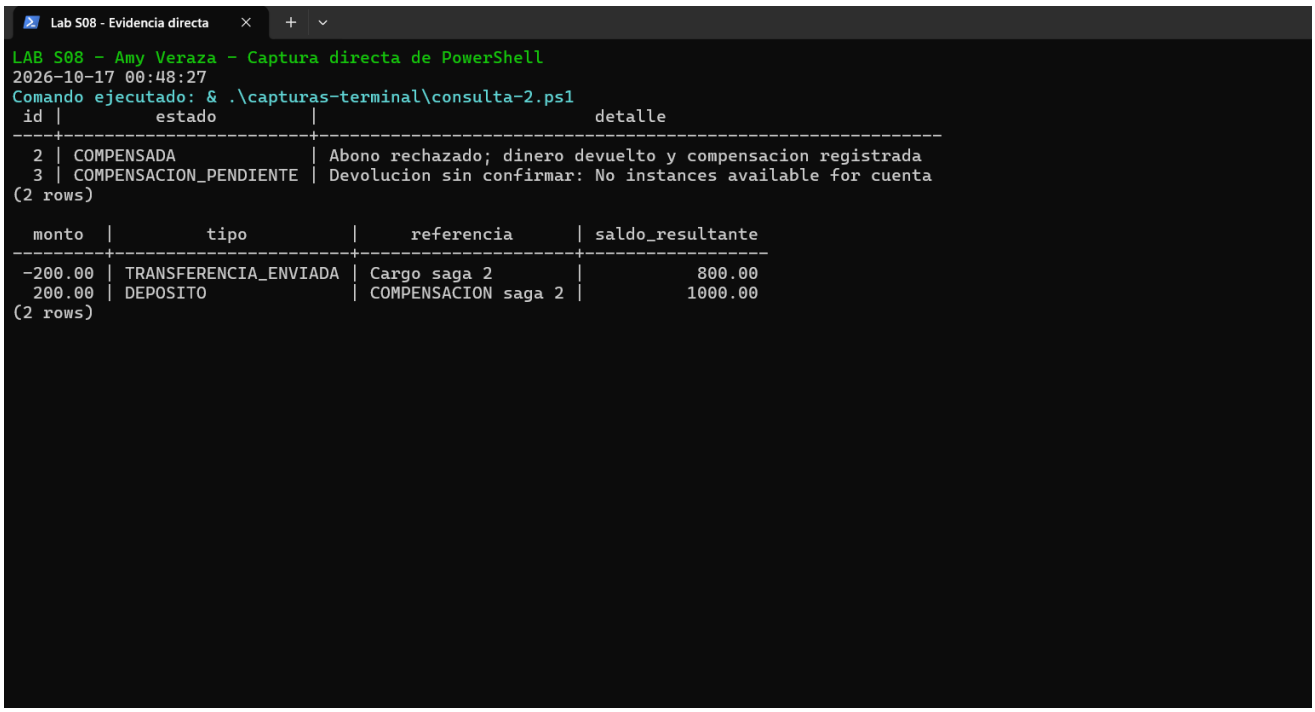
paso() guarda estado y detalle. Uso FALLA_ABONO_CLABE para rechazar el abono antes de enviarlo: asi se sabe que destino no recibio el dinero. Un timeout o un 5xx no dan esa certeza, por lo que no devuelvo a ciegas ni reintento los POST automaticamente. El asiento de devolucion es DEPOSITO positivo con descripcion COMPENSACION saga <id>.

4. Compensacion comprobada

Con FALLA_ABONO_CLABE=002187000000004082, origen y destino comenzaron en 1000 y 500. Tras intentar transferir 200, ambos conservaron esos saldos. La respuesta HTTP 201 contiene estado COMPENSADA y explica la devolucion. 201 indica que se creo el registro; no equivale a que destino recibio el monto.

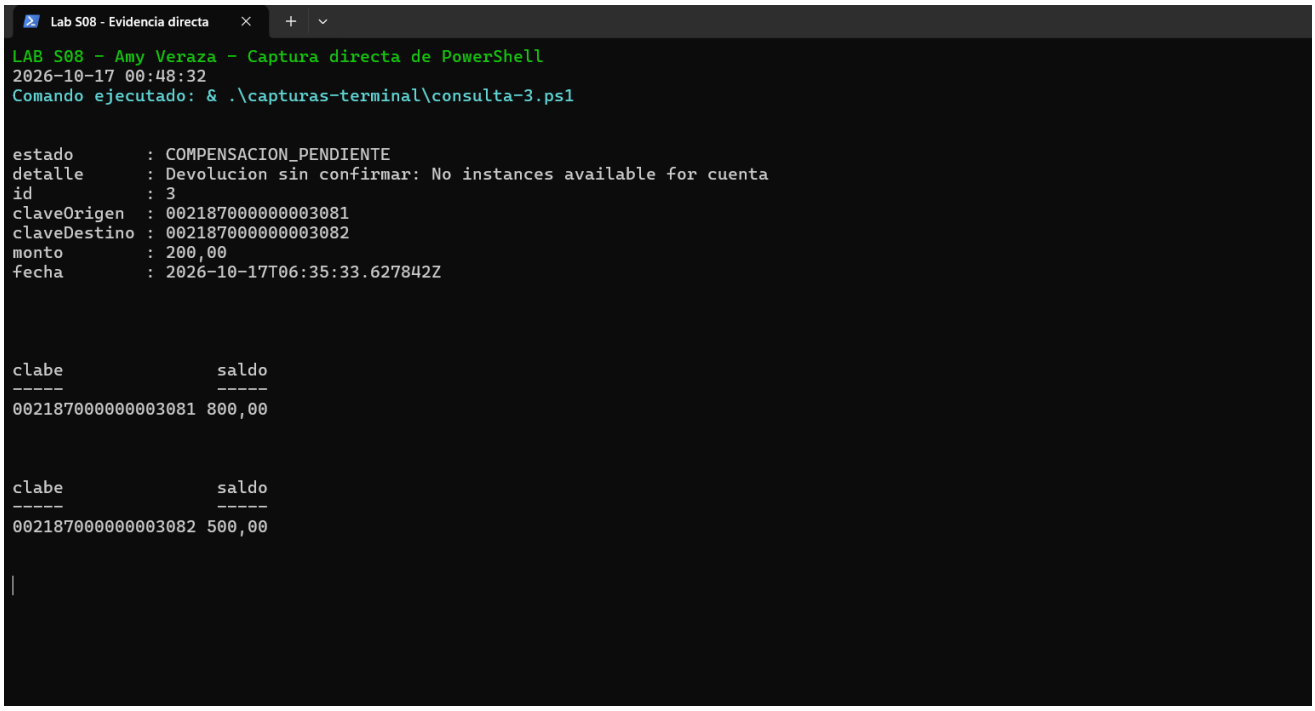


Consulta real del estado final y del ledger:



5. Si la devolucion tambien falla

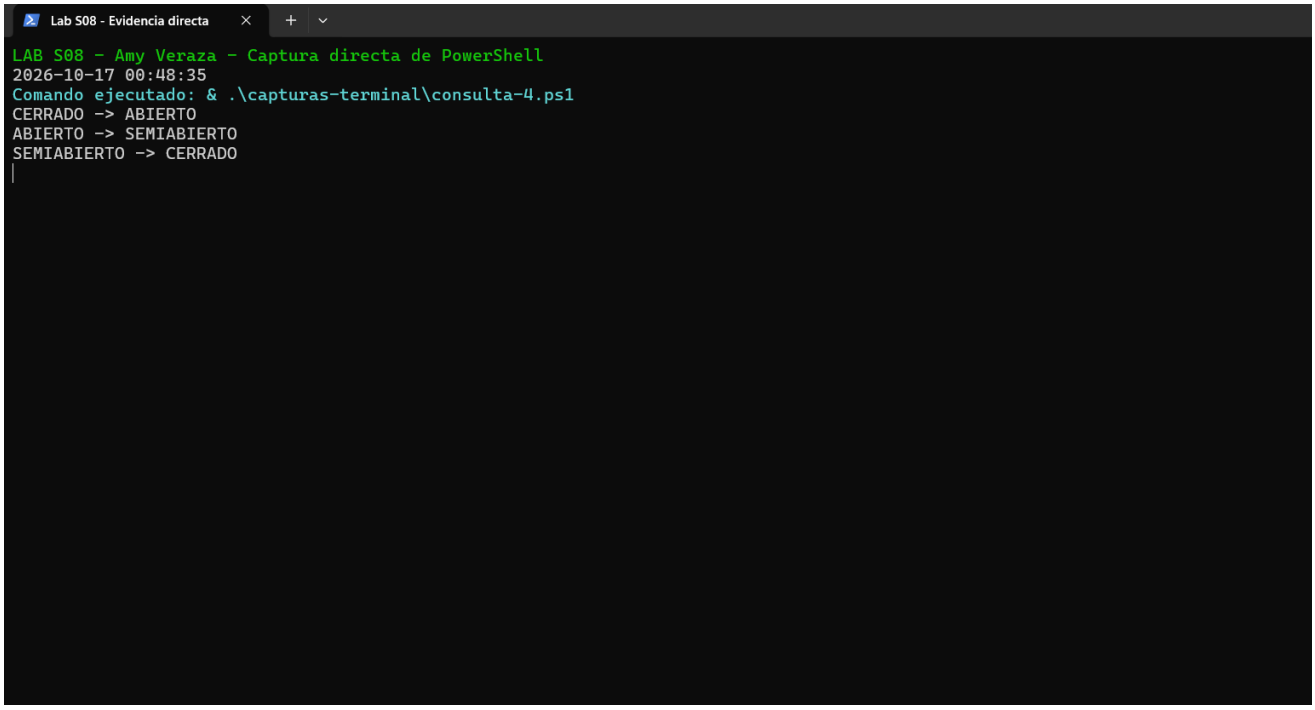
Active la falla para destino 002187000000003082 y una pausa de 20 segundos. Con cuenta todavia disponible, la consulta durante la pausa mostro el origen cargado. Despues apague cuenta: la devolucion no pudo confirmarse. La saga guardo COMPENSACION_PENDIENTE, sin fingir que se completo.

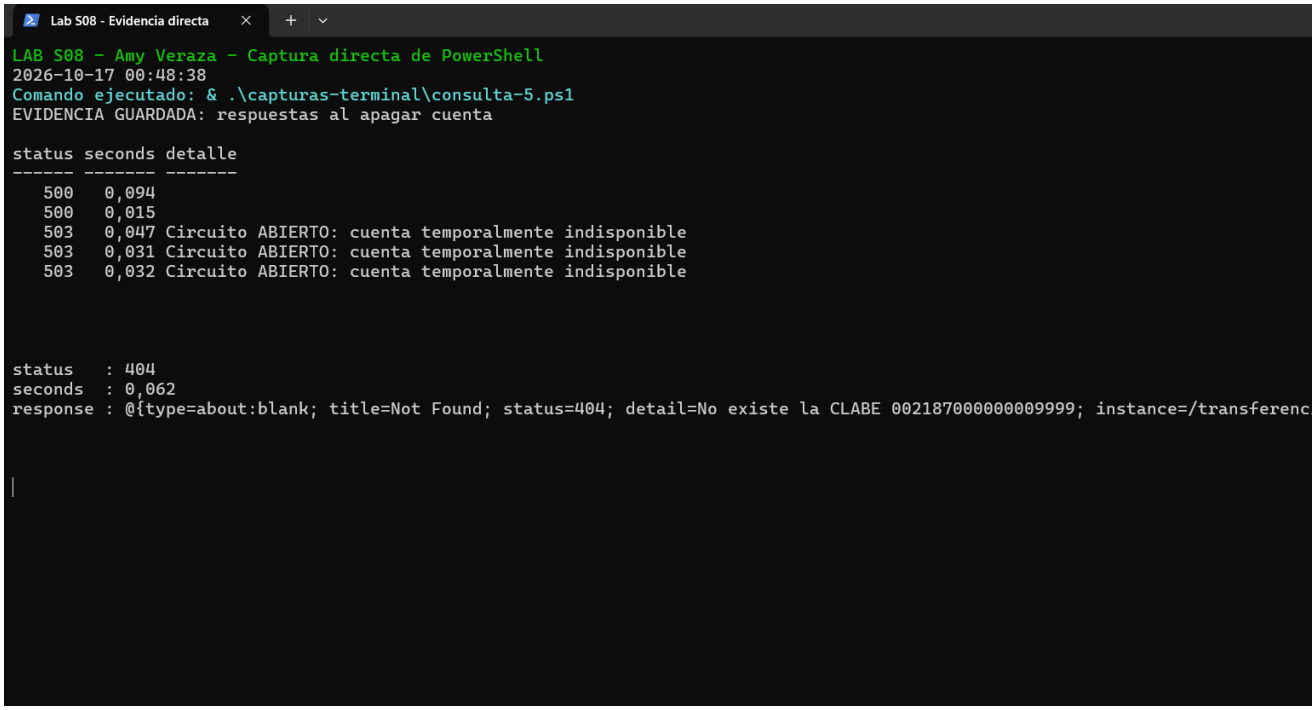


Para terminarla hace falta conciliar el destino y ofrecer idempotencia en cuenta: una clave saga:<id>:compensacion, con unicidad, resultado y cambio de saldo confirmados en la misma transaccion. Entonces un trabajador puede reclamar pendientes y reintentar con espera creciente. Una marca solo en transferencia no basta: si la respuesta de la devolucion se pierde, volver a abonarla duplicaria dinero. Tambien deben deduplicarse los movimientos.

6. Circuit breaker manual

CircuitoCuenta permite llamadas en CERRADO. Tras tres fallas de infraestructura pasa a ABIERTO y rechaza con 503 sin hacer HTTP. Despues de diez segundos admite un solo sondeo SEMIABIERTO. Si responde, vuelve a CERRADO; si falla, reabre. Los 4xx de negocio no cuentan como caida. Configuro conexion de 2 segundos y lectura de 5 segundos en ClientesHttp.





Limitacion observada: antes de abrir, las primeras dos peticiones con el directorio sin instancias devolvieron 500 por No instances available. El circuito conto esas fallas y las siguientes devolvieron 503 con detalle Circuito ABIERTO. Mejoraria el manejador de esa excepcion para responder 503 tambien antes de abrir. Se conservan esas respuestas en la evidencia.

El sondeo usa un destino inexistente: obtener 404 confirma que cuenta ya responde sin ejecutar un cargo nuevo. Los logs adjuntos muestran la transicion de semiabierto a cerrado. La transferencia pendiente sigue pendiente: recuperar conectividad no compensa dinero automaticamente.

7. Lo que la saga no da

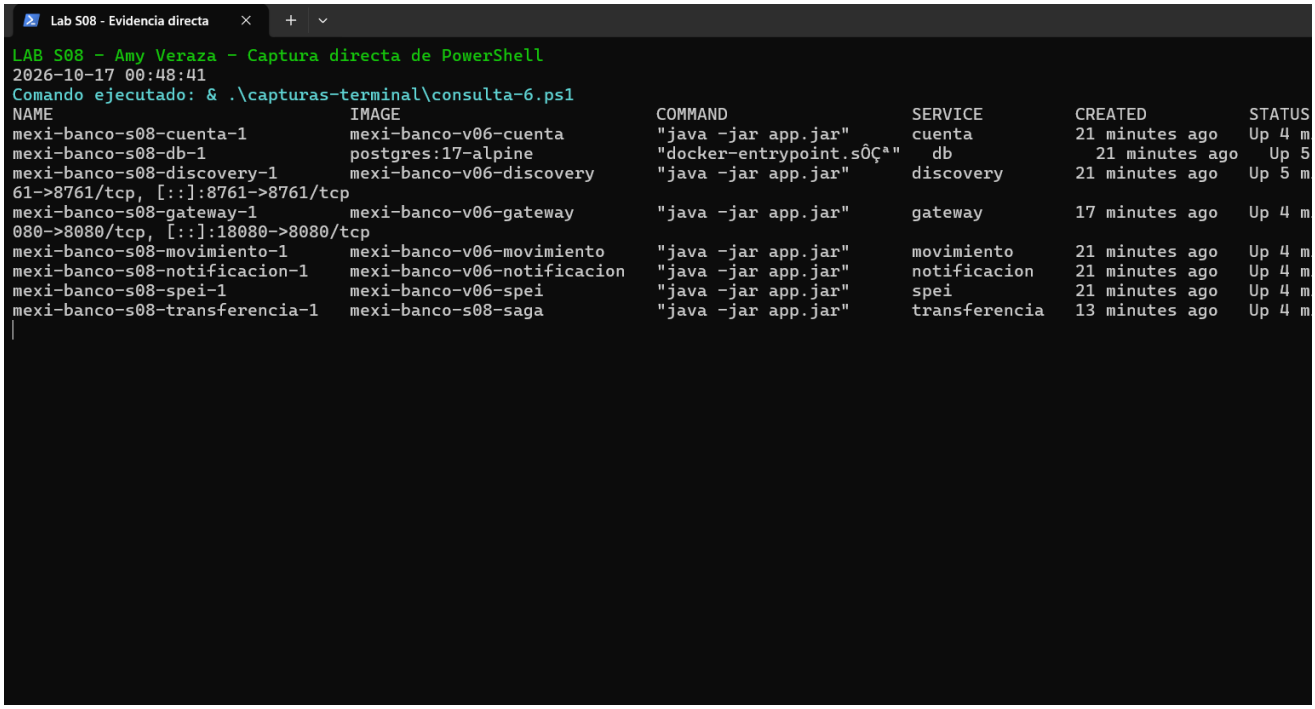
No existe aislamiento global. Entre el cargo y la devolucion, otra persona puede consultar el origen y ver 800, aunque al final vuelva a 1000. Incluso puede tomar decisiones usando ese saldo intermedio. Compensar no borra esas observaciones ni las acciones de otros servicios. Para controlarlas harian falta reservas y reglas adicionales.

Validacion y reproduccion

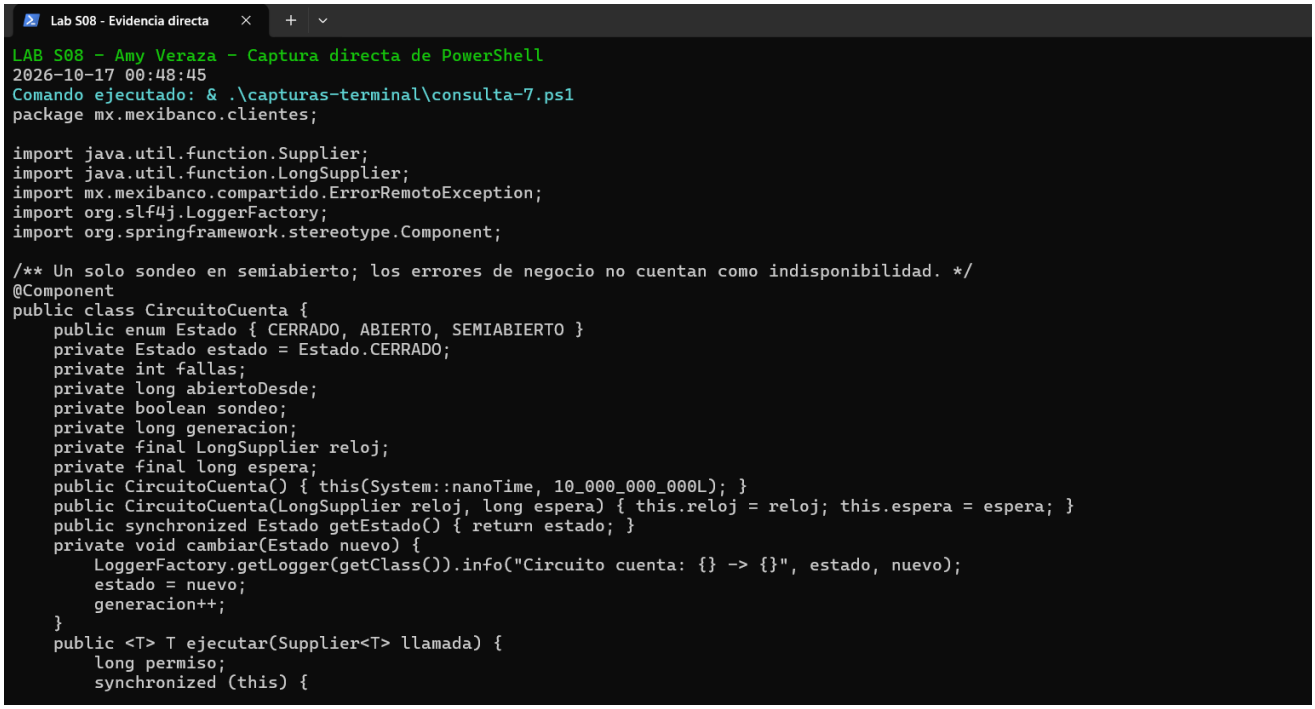
La suite local previa registra 11 pruebas, cero fallas y cero errores; se adjunta el log original. Esta entrega agrega evidencia real HTTP, SQL y logs en Docker. Se reutilizaron las imagenes locales de la base S07 y mexi-banco-s08-saga; el README documenta como reconstruir desde las fuentes entregadas. No se implemento reintento monetario automatico porque cuenta aun no ofrece idempotencia.

Fuente del enunciado:
https://github.com/OscarRuiz21/sd-2027-1/blob/main/labs/s08-saga-circuit-breaker/README.md

Capturas adicionales: contenedores y codigo



```
Lab S08 - Evidencia directa
LAB S08 - Amy Veraza - Captura directa de PowerShell
2026-10-17 00:48:41
Comando ejecutado: & .\capturas-terminal\consulta-6.ps1
NAME                                IMAGE                          COMMAND                  SERVICE         CREATED          STATUS
mexi-banco-s08-cuenta-1             mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          21 minutes ago   Up 4 m
mexi-banco-s08-db-1                 postgres:17-alpine             "docker-entrypoint.sÔÇ¦"   db              21 minutes ago   Up 5
mexi-banco-s08-discovery-1          mexi-banco-v06-discovery       "java -jar app.jar"      discovery       21 minutes ago   Up 5 m
61->8761/tcp, [::]:8761->8761/tcp
mexi-banco-s08-gateway-1            mexi-banco-v06-gateway         "java -jar app.jar"      gateway         17 minutes ago   Up 4 m
080->8080/tcp, [::]:18080->8080/tcp
mexi-banco-s08-movimiento-1         mexi-banco-v06-movimiento      "java -jar app.jar"      movimiento      21 minutes ago   Up 4 m
mexi-banco-s08-notificacion-1       mexi-banco-v06-notificacion    "java -jar app.jar"      notificacion    21 minutes ago   Up 4 m
mexi-banco-s08-spei-1               mexi-banco-v06-spei            "java -jar app.jar"      spei            21 minutes ago   Up 4 m
mexi-banco-s08-transferencia-1      mexi-banco-s08-saga            "java -jar app.jar"      transferencia   13 minutes ago   Up 4 m
```



```
Lab S08 - Evidencia directa
LAB S08 - Amy Veraza - Captura directa de PowerShell
2026-10-17 00:48:45
Comando ejecutado: & .\capturas-terminal\consulta-7.ps1
package mx.mexibanco.clientes;

import java.util.function.Supplier;
import java.util.function.LongSupplier;
import mx.mexibanco.compartido.ErrorRemotoException;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

/** Un solo sondeo en semiabierto; los errores de negocio no cuentan como indisponibilidad. */
@Component
public class CircuitoCuenta {
    public enum Estado { CERRADO, ABIERTO, SEMIABIERTO }
    private Estado estado = Estado.CERRADO;
    private int fallas;
    private long abiertoDesde;
    private boolean sondeo;
    private long generacion;
    private final LongSupplier reloj;
    private final long espera;
    public CircuitoCuenta() { this(System::nanoTime, 10_000_000_000L); }
    public CircuitoCuenta(LongSupplier reloj, long espera) { this.reloj = reloj; this.espera = espera; }
    public synchronized Estado getEstado() { return estado; }
    private void cambiar(Estado nuevo) {
        LoggerFactory.getLogger(getClass()).info("Circuito cuenta: {} -> {}", estado, nuevo);
        estado = nuevo;
        generacion++;
    }
    public <T> T ejecutar(Supplier<T> llamada) {
        long permiso;
        synchronized (this) {
```